

CSC 3204: AUTOMATA THEORY

Lecture XI: Regular and Non-Regular Languages

1

Definition – Regular Languages

- Any language that can be defined using a regular expression is called a **regular language**.
- Are all languages regular? *No!*
- So what are the properties of regular languages?

2

Theorem 10

- If L_1 and L_2 are regular languages
Then so are $L_1 + L_2$, $L_1 L_2$ and L_1^*
 - $L_1 + L_2$ means the language of all words in either L_1 or L_2
 - $L_1 L_2$ means the language of all words formed by concatenating a word from L_1 with a word from L_2
 - L_1^* means the concatenation of arbitrarily many factors from L_1
- The result stated in theorem 10 is closed under union, concatenation, and the Kleene star operator

3

Theorem 10: Proof by Regex

- If L_1 is a regular language then it is defined by a regular expression termed r_1
- If L_2 is a regular language then it is defined by a regular expression termed r_2
- Given these two regular expressions we can also define
 - $L_1 + L_2$ using $r_1 + r_2$
 - $L_1 L_2$ using $r_1 r_2$
 - L_1^* using $(r_1)^*$

4

Theorem 10: Proof by Machines

- A very similar proof to the one on the previous page can be performed using transition graphs.
- How?

5

Definition

- If L is a language over the alphabet Σ , we define its complement, L' , to be the language of all strings of letters from Σ that are not words in L .

6

Theorem 11

- If L is a regular language, then L' is also a regular language
- In other words the set of regular languages is closed under complementation

7

Proof of Theorem 11

- If L is a regular language, then we know from Kleene's theorem that there is some FA that accepts the language L
- Some of the states in this FA are final states and, most likely, some are not.
- The implementation of this proof requires us to reverse the 'final' status of each of the states in the accepting FA.
 - i.e. if a state was final in the original FA it must become 'nonfinal' (in the complement FA), and if it was 'nonfinal' then it must become final.

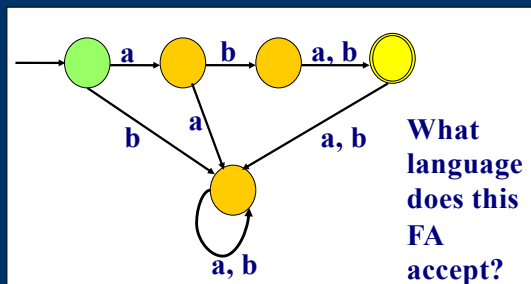
8

Proof of Theorem 11 (cont.)

- The result of this is that if an input string formerly ended in a final state, it will now end in a 'nonfinal' state and be rejected, while,
- An input string that formerly ended in a 'nonfinal' state will now end up in a final state and be accepted!
- Therefore, the complementary FA, FA' accepts all input strings that were rejected by FA , and rejects all input strings that were accepted by FA
- Therefore FA' accepts exactly the language L'
- So, by Kleene's Theorem L' is regular

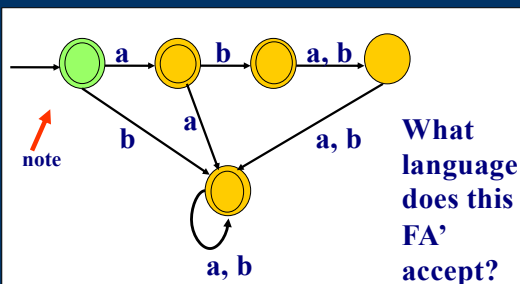
9

Theorem 11: Illustrated



10

Theorem 11: Illustrated cont...



11

Theorem 12

- If L_1 and L_2 are regular languages, then $L_1 \cap L_2$ is also a regular language
- In other words, the set of regular languages is closed under intersection

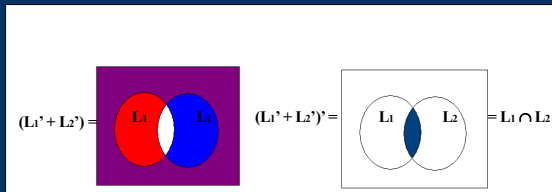
12

Proof of Theorem 12

- By DeMorgan's Law for sets of any kind (regular languages or not):

$$L_1 \cap L_2 = (L_1' + L_2')'$$

Illustrating using Venn Diagrams:



13

Proof of Theorem 12 (cont'd)

- This means that the language $L_1 \cap L_2$ consists of all words that are not in either L_1' or L_2' .
- Since L_1 and L_2 are regular, then so is $(L_1' + L_2')$.
- If $(L_1' + L_2')$ is regular then so is $(L_1' + L_2')'$.
- Therefore $L_1 \cap L_2$ is regular.

14

Summary

- Any language that can be defined using a regular expression is regular
- Not all languages are regular (as we shall soon illustrate)
- Regular languages are closed under choice (union), concatenation, closure, complementation and intersection

15

Non-regular Languages

16

Recap: Regular Languages

- Regular languages have many different forms and structures, but they have some regularities:
 - Languages with some required substrings
 - Languages that forbid some substrings
 - Languages that begin or end with certain strings
 - Languages with certain even / odd properties
 - Etc.

17

Definition : Non-Regular Languages

- A language that cannot be defined by a regular expression is called a **nonregular language**.
- By Kleene's theorem nonregular languages cannot be defined by TGs, FAs or any other regular automaton

18

Nonregular Languages

- Examples of Nonregular languages:
 - $L = \{a^n b^n\}$
 - Palindrome
 - Prime

Why are these languages non-regular?

19

Non-regular Languages

- Examples of Non-regular languages:
 - $L = \{a^n b^n\}$
 - Palindrome
 - Prime

Why are these languages nonregular?

- A very simple explanation of why these languages are non-regular involves the 'pumping lemma'
- Remember!** An automaton must accept ALL strings that belong to the language and reject all strings that do not belong to the language.

20

Why non-regular (cont'd)

- It is not possible to develop an automaton that accepts all the possible strings without creating a circuit (or loop) in the machine
- Once you create a circuit you cannot control how many times the loop is 'pumped' (traversed) and therefore it becomes possible to accept strings that do not belong to the language!

21

Pumping Lemma

22

Non-regular languages $\{a^n b^n : n \geq 0\}$
 $\{vv^R : v \in \{a,b\}^*\}$

Regular languages

a^*b $b^*c + a$
 $b + c(a + b)^*$
 etc...

23

How can we prove that a language L is not regular?

Prove that there is no DFA or NFA or RE that accepts L

Difficulty: this is not easy to prove
 (since there is an infinite number of them)

Solution: use the Pumping Lemma !!!

24

The Pigeonhole Principle

25

4 pigeons

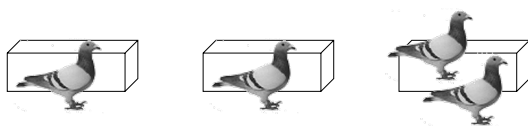


3 pigeonholes



26

A pigeonhole must contain at least two pigeons



27

n pigeons



m pigeonholes

$n > m$



28

The Pigeonhole Principle

n pigeons

m pigeonholes

$n > m$

There is a pigeonhole with at least 2 pigeons



29

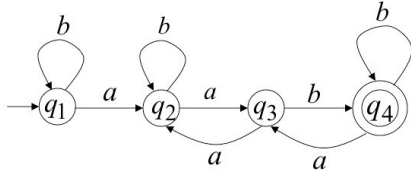
The Pigeonhole Principle

and

DFAs

30

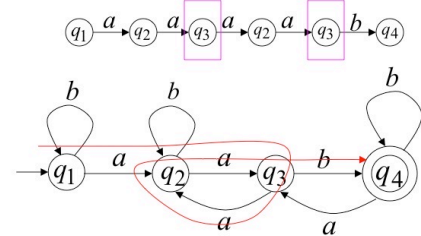
Consider a DFA with 4 states



31

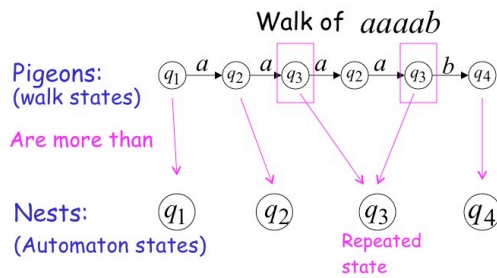
Consider the walk of a "long" string $aaaaab$
(length at least 4)

A state is repeated in the walk of $aaaaab$



32

The state is repeated as a result of
the pigeonhole principle

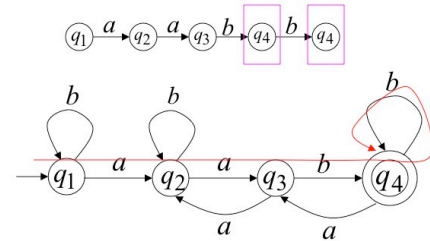


33

Consider the walk of a "long" string: $aabb$
(length at least 4)

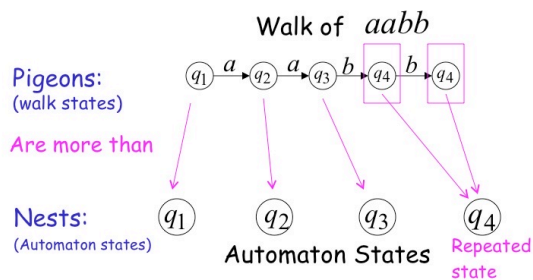
Due to the pigeonhole principle:

A state is repeated in the walk of $aabb$



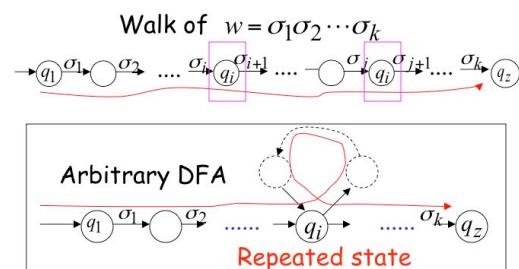
34

The state is repeated as a result of
the pigeonhole principle

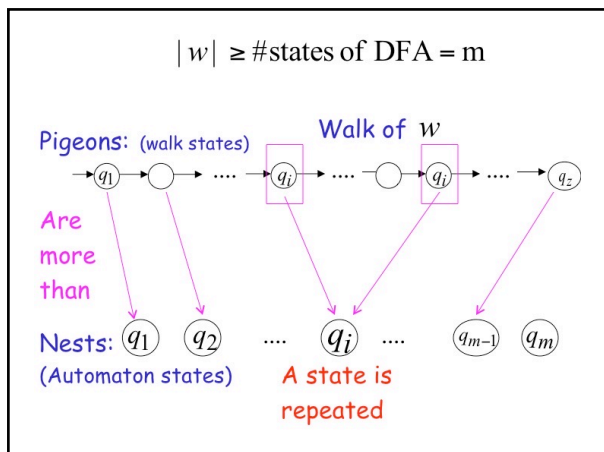


35

In General: If $|w| \geq \# \text{states of DFA}$,
by the pigeonhole principle,
a state is repeated in the walk w



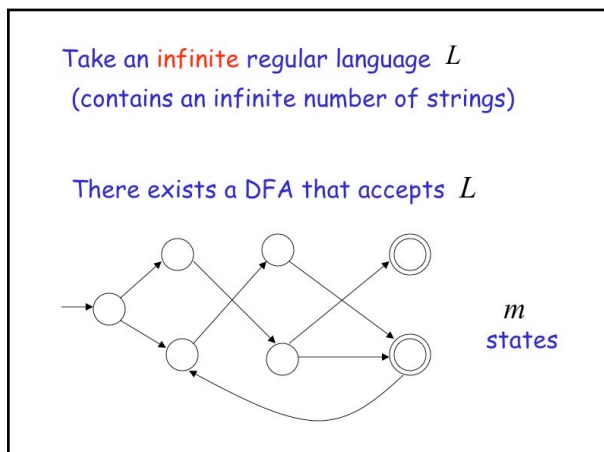
36



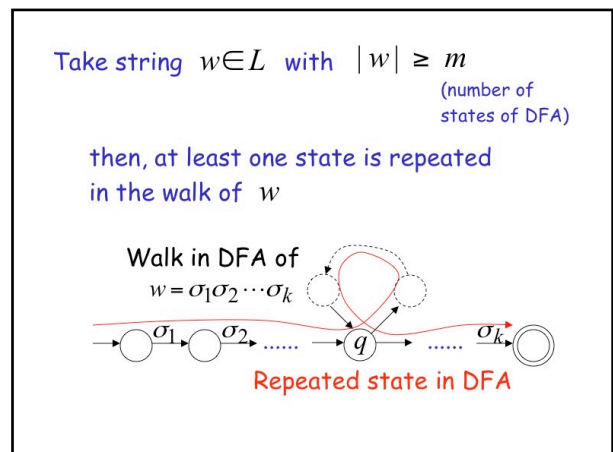
37

The Pumping Lemma

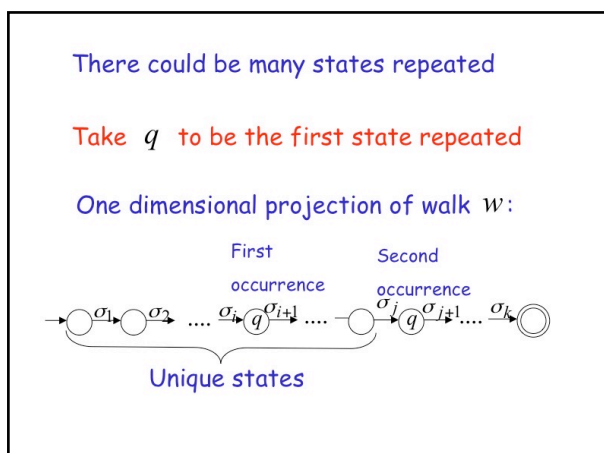
38



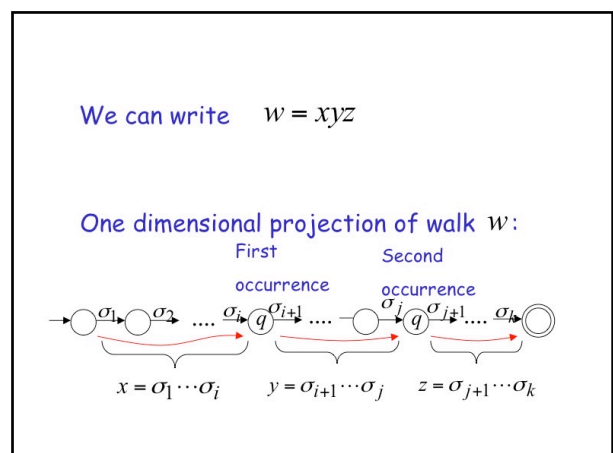
39



40

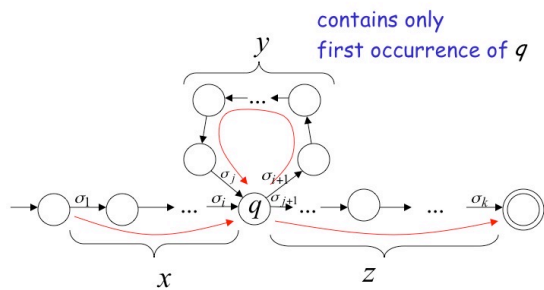


41



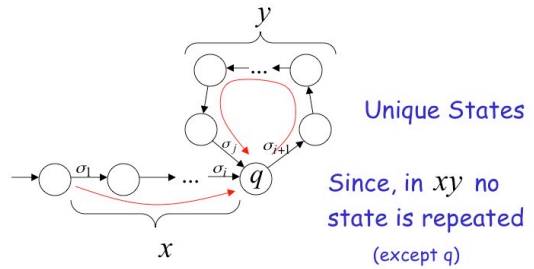
42

In DFA: $w = x y z$



43

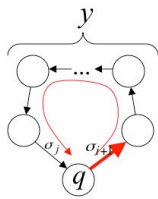
Observation: $\text{length } |x y| \leq m$ number of states of DFA



44

Observation: $\text{length } |y| \geq 1$

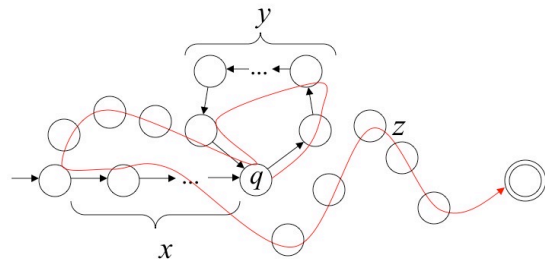
Since there is at least one transition in loop



45

We do not care about the form of string z

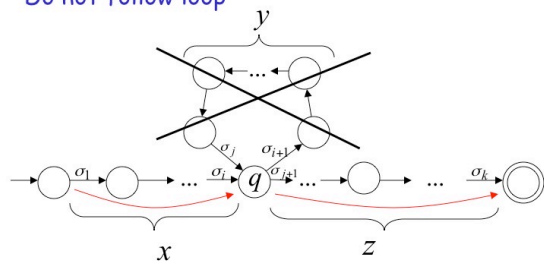
z may actually overlap with the paths of x and y



46

Additional string: The string $x z$ is accepted

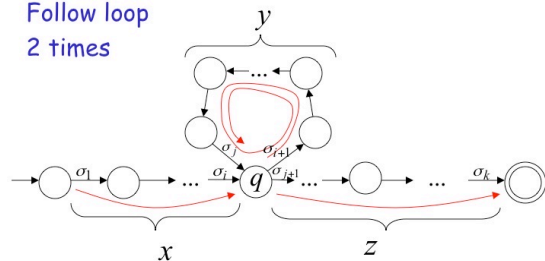
Do not follow loop



47

Additional string: The string $x y y z$ is accepted

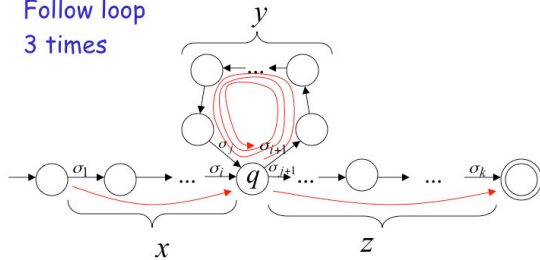
Follow loop 2 times



48

Additional string: The string $x y y y z$ is accepted

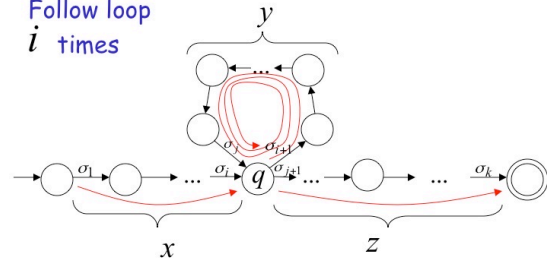
Follow loop
3 times



59

In General: The string $x y^i z$ is accepted $i = 0, 1, 2, \dots$

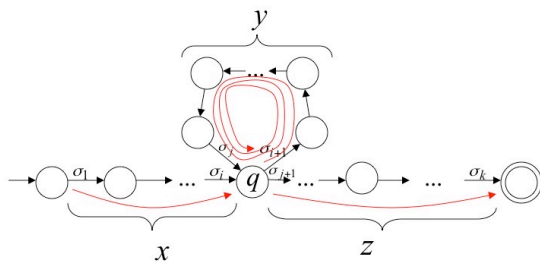
Follow loop
 i times



50

Therefore: $x y^i z \in L$ $i = 0, 1, 2, \dots$

Language accepted by the DFA



51

We've just described the
pumping lemma!

52

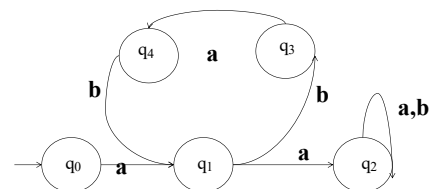
The Pumping Lemma:

- Given a infinite regular language L
- there exists an integer m (critical length)
- for any string $w \in L$ with length $|w| \geq m$
- we can write $w = x y z$
- with $|x y| \leq m$ and $|y| \geq 1$
- such that: $x y^i z \in L$ $i = 0, 1, 2, \dots$

53

Ex. Pumping Lemma of a regular language

consider the language $L = L(R)$ where
 $R = a(bab)^*a(a+b)^*$



x = walk of any string before the loop (q_0, q_1) = 'a'
 y = walk of a string within the loop (q_1, q_3, q_4, q_1) = 'bab'
 z = walk of any string after the loop (q_1, q_2) = $a(a+b)^*$

54

Ex. Pumping Lemma of a regular language

consider the language $L = L(R)$ where
 $R = a(bab)^*a(a+b)^*$

By pumping lemma, $W = XY^iZ$ for $i=0,1,2,3,\dots$

Where $M = 5$, $|XY| \leq 5$ and $|Y| \geq 1$

$X = 'a'$, $Y = 'bab'$ and $Z = a(a+b)^*$

If $i = 0$, $W = xy^0z = a(a+b)^*$ which is in L

If $i = 1$, $W = xy^1z = ababa(a+b)^*$ which is in L

If $i = 2$, $W = xy^2z = ababbaba(a+b)^*$ which is in L

...

If $i = 5$, $W = xy^5z = ababbabbabbaba(a+b)^*$ which is in L

...

Pumping lemma holds true for $W = XY^iZ$ for any i , since L is a regular infinite language.

55

Applications of the Pumping Lemma

56

Observation:

Every language of finite size has to be regular

(we can easily construct an NFA
that accepts every string in the language)

Therefore, every non-regular language
has to be of infinite size

(contains an infinite number of strings)

57

Suppose you want to prove that
An infinite language L is not regular

1. Assume the opposite: L is regular
2. The pumping lemma should hold for L
3. Use the pumping lemma to obtain a contradiction
4. Therefore, L is not regular

58

Explanation of Step 3: How to get a contradiction

1. Let m be the critical length for L
2. Choose a particular string $w \in L$ which satisfies the length condition $|w| \geq m$
3. Write $w = xyz$
4. Show that $w' = xy^iz \notin L$ for some $i > 1$
Usually sufficient for $i = 2$
5. This gives a contradiction, since from pumping lemma $w' = xy^iz \in L$

59

Note: It suffices to show that
only one string $w \in L$
gives a contradiction

You don't need to obtain
contradiction for every $w \in L$

60

Example of Pumping Lemma application

Theorem: The language $L = \{a^n b^n : n \geq 0\}$ is not regular

Proof: Use the Pumping Lemma

61

$$L = \{a^n b^n : n \geq 0\}$$

Assume for contradiction that L is a regular language

Since L is infinite we can apply the Pumping Lemma

62

$$L = \{a^n b^n : n \geq 0\}$$

Let m be the critical length for L

We Pick a string w such that: $w \in L$
and length $|w| \geq m$

We pick $w = a^m b^m$

63

From the Pumping Lemma:

we can write $w = a^m b^m = x y z$

with lengths $|x y| \leq m$, $|y| \geq 1$

$$w = xyz = a^m b^m = \overbrace{a \dots a}^m \overbrace{a \dots a}^m \overbrace{ab \dots b}^m$$

$\underbrace{\hspace{1cm}}_x \quad \underbrace{\hspace{1cm}}_y \quad \underbrace{\hspace{1cm}}_z$

Thus: $y = a^k$, $1 \leq k \leq m$

64

$$x y z = a^m b^m \quad y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^i z \in L$
 $i = 0, 1, 2, \dots$

Thus: $x y^2 z \in L$

65

$$x y z = a^m b^m \quad y = a^k, \quad 1 \leq k \leq m$$

From the Pumping Lemma: $x y^2 z \in L$

$$x y^2 z = \overbrace{a \dots a}^{m+k} \overbrace{a \dots a}^m \overbrace{ab \dots b}^m$$

$\underbrace{\hspace{1cm}}_x \quad \underbrace{\hspace{1cm}}_y \quad \underbrace{\hspace{1cm}}_y \quad \underbrace{\hspace{1cm}}_z$

Thus: $a^{m+k} b^m \in L$

66

$$a^{m+k}b^m \in L \quad k \geq 1$$

BUT: $L = \{a^n b^n : n \geq 0\}$



$$a^{m+k}b^m \notin L$$

CONTRADICTION!!!

67

Therefore: Our assumption that L is a regular language is not true

Conclusion: L is not a regular language

END OF PROOF

68

Non-regular language $\{a^n b^n : n \geq 0\}$

Regular languages

$$L(a^* b^*)$$

69

Example of Pumping Lemma application

Theorem: The language $\{vv^R : v \in \{a,b\}^*\}$ is not regular

Proof: Use the Pumping Lemma to prove That the above language is not Regular.

70